



中山大學
SUN YAT-SEN UNIVERSITY

数据库原理实验报告

二次元周边商品店数据库设计与实现

姓 名 郑立峰 李维政

学 号 24325341 24325143

学 院 计算机学院

专 业 计算机科学与技术

2026 年 10 月 4 日

目录

1 实验目标与阶段成果	1
2 经营场景与业务需求	1
2.1 场景选择与数据边界	1
2.2 角色职责与关键业务约定	2
2.3 现货、预售与补货流程	2
3 关系模式与典型设计	3
3.1 实体及数据表总体结构	3
3.2 商品设计与作品角色一致性	3
3.3 销售订单与明细的统一结构	5
3.4 库存状态与预售决策数据	5
4 数据库建立与数据修改实验	6
4.1 执行顺序与样例数据	6
4.2 典型 CRUD 流程与结果	6
5 经营查询与统计视图	7
5.1 查询覆盖与统计口径	7
5.2 连接与聚合的典型分析	7
5.3 视图封装与复用	9
6 完整性约束与正反例验证	10
6.1 约束总体设计与测试方法	10
6.2 典型正反例及结果分析	10
7 角色权限与正常越权测试	11
7.1 最小权限与角色映射	11
7.2 典型权限验证	12
8 小组协作与 AI 辅助下的独立判断	13
8.1 成员分工与协同方式	13
8.2 AI 建议、人工选择与验证	14
8.3 整合审查与改进	14
9 实验总结与后续工作	15
相关材料索引	15

1 实验目标与阶段成果

本阶段围绕一家小型线下二次元周边商品店，完成从经营需求到关系数据库原型的设计与实现。我们将商品、库存、会员、员工、销售、预售及采购联系起来，在 SQL Server 中建立数据库，并通过增删改查、经营统计、完整性约束和角色授权验证设计的可用性。阶段版本为 v0.1，重点是形成能够按脚本复现的数据库基础，为后续数据库应用、编程和事务实验提供统一的数据结构。

周次	实验主题	本组主要成果
第一周	经营需求与数据边界	确定单门店场景、参与者、现货与预售流程及简化规则。
第二周	关系模式设计	设计 15 张表，明确属性、域、主码、候选码、外码及业务样例。
第三周	DDL 与数据修改	完成建库建表、样例装载及商品、库存、销售订单 CRUD。
第四周	查询、视图、约束与授权	完成 9 条经营查询、4 个视图、完整性正反例和 3 种员工角色权限测试。

表 1: 第 1 至 4 周实验任务与阶段成果

实验在 Windows 环境中使用 Microsoft SQL Server 和 SQL Server Management Studio (SSMS) 完成，截图所示本地实例版本为 17.0.1135，数据库名为 AnimeGoodsStoreDB。我们在 SSMS 中按编号执行 00–07 脚本，并由组员检查运行结果、业务含义和文档说明。报告围绕总体结构与典型案例展开，各表完整字段定义和完整 SQL 保留在协同仓库中，便于进一步查阅和复现。

2 经营场景与业务需求

2.1 场景选择与数据边界

我们选择徽章、亚克力立牌、挂件、色纸及玩偶等商品构成的线下周边店。相比线上商城或连锁经营，单门店能够清楚体现销售、库存和员工职责，又不必引入配送、跨店调拨等额外流程。商品的作品/IP、具体角色和预售活动为常规零售场景提供了特色，也为后续销量分析与补货决策留下扩展空间。

进入数据库的信息包括商品类别与资料、IP 及角色、当前库存、会员、员工、供应商、销售订单与明细、预售活动与预订、采购订单与明细。它们分别服务于商品识别、交易记录、需求汇总和采购执行。普通非会员顾客不建立个人档案，其订单的会员字段为空；供应商虽然需要存档，但不登录店铺系统。线上商城、物流、售后退换货、多门店、复杂营销和自动推荐暂不纳入本阶段，避免业务范围超出课程进度。

2.2 角色职责与关键业务约定

角色或参与者	职责与定位
店长/管理员	管理商品、员工与供应商；开展预售，确定采购及额外备货；查看经营统计。
销售/收银店员	查询商品和可售库存，处理会员与销售订单，确认预售提货。
库存管理员	到货验收、库存盘点、库存维护及登记采购执行结果。
会员	购买现货、参加预售、取消预订及提货；后续通过应用访问本人数据。
普通顾客、供应商	分别参与现货购买和供货，不直接拥有数据库操作账号。

表 2: 业务参与者与职责划分

小组讨论后确定三项简化约定：一个商品最多属于一个 IP、对应一个具体角色，群像商品不指定具体角色；一次预订整体取消或整体提货，不支持部分处理；一条预订最多对应一笔预售提货订单，不合并多个活动的预订结账。这些约定保留了预售的业务特点，同时使关系结构和实验操作保持可解释、可实施。

2.3 现货、预售与补货流程

现货业务从查询可售库存开始，随后创建订单和明细、确认数量与成交价、完成结账，并维护库存。库存达到或低于警戒值时，由库存管理员发现补货需求，店长确定采购量，供应商到货后再验收入库。

预售则先由店长设置活动时间、预售价和订金，会员预订并支付订金，截止后统一汇总需求和确定采购。截止前取消退还订金，截止后仍可自行取消但订金不退；到货后有效预订对应商品被预留，会员在期限内补尾款并整体提货。采购后取消、逾期未提货或额外备货产生的剩余商品进入普通现货销售。因此，**预售与现货是同一商品的两种销售方式，不是两个固定商品类别。**

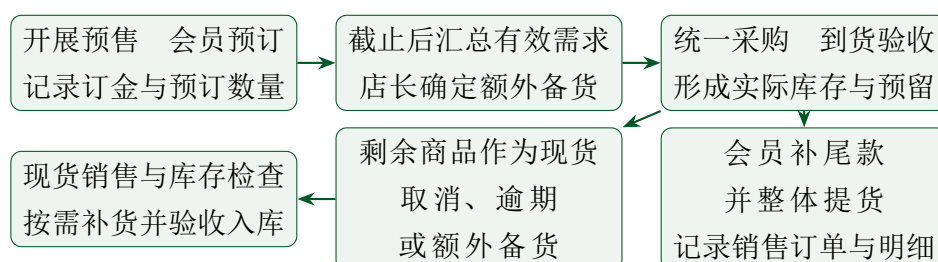


图 1: 预售与现货之间的经营流程关系

这些流程用于指导表结构和样例组织。本阶段已实现数据存储、查询和声明式约束；自动库存联动、跨表退款判断及完整状态迁移计划结合后续存储过程和事务实现。详细业务说明见《第一周业务设计.md》。

3 关系模式与典型设计

3.1 实体及数据表总体结构

15 张表按业务职责组织，而不是对每一种操作单独建表。商品基础资料提供分类维度，销售与采购分别采用“订单头+明细”结构，预售活动与预订连接商品和会员，库存保存当前经营状态。

模块	数据表	核心用途
商品基础资料	ProductCategory、AnimeIP、CharacterInfo、Product	分类、作品、角色与商品资料。
人员与供应商	Member、BusinessRole、Employee、Supplier	会员、经营职责、员工和供货来源。
库存	Inventory	实际库存、预留库存与补货警戒值。
销售	SalesOrder、SalesOrderItem	现货与预售提货的统一交易记录。
预售	PresaleActivity、PresaleReservation	活动安排、需求、订金与预订状态。
采购	PurchaseOrder、PurchaseOrderItem	普通补货与预售采购及验收数量。

表 3: 实体及数据表清单

图2突出主要数据关联：销售和采购通过明细连接商品；预订连接活动与会员；销售订单还可关联预订。员工资料内部由Employee.role_id 引用BusinessRole，预售创建者、采购经办与验收员工也关联员工表。各表完整字段、关系模式和样例元组详见《第二周数据库设计.md》，以下选择典型结构解释设计理由。

3.2 商品设计与作品角色一致性

Product 使用内部编号product_id 作为主码，业务编号product_code 设置唯一约束作为候选码。二者分别承担表间引用与业务识别，商品名称不要求唯一，避免不同

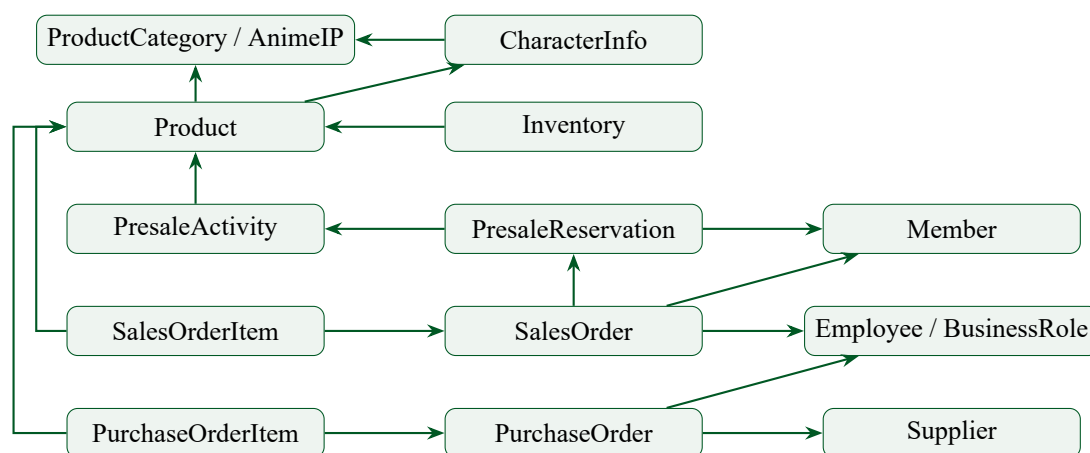


图 2: 主要关系概览，箭头由引用方指向被引用方；基础资料和员工资料合并展示

款式同名时无法录入。

典型字段	类型及约束	设计含义
product_id	INT, PK, 正整数	稳定的内部标识。
product_code	VARCHAR(20), UNIQUE	商品业务编号。
category_id	INT, 非空外码	每件商品必须有类别。
ip_id、character_id	INT, 可空外码	区分无明确 IP、群像及具体角色商品。
sale_price	DECIMAL(10,2), 大于 0	当前售价，以定点数保存金额。
product_status	VARCHAR(20), CHECK	限定 PENDING、ON_SALE、STOPPED。

表 4: 商品表的典型字段设计

中文名称使用 NVARCHAR，业务编号使用 VARCHAR，金额使用 DECIMAL，业务时间使用 DATETIME2；空值用于表达确实不存在或尚未确定的信息，不用随意的字符串占位。原神群像徽章可以只填 IP，而甘雨立牌同时关联 IP 与角色。角色不为空时必须指定 IP，再由复合外键保证两者属于同一条角色记录：

```
CHECK (character_id IS NULL OR ip_id IS NOT NULL),
FOREIGN KEY (character_id, ip_id)
REFERENCES dbo.CharacterInfo(character_id, ip_id)
```

仅设置两个独立外键只能保证“IP 存在”和“角色存在”，不能排除“原神+三月七”的错误组合。为支持复合引用，角色表另设 `UNIQUE(character_id,ip_id)`；该组合包含已有主码，是辅助引用的唯一约束，不是新的最小候选码。角色名称真正的业务候选码是 `(ip_id,character_name)`。

3.3 销售订单与明细的统一结构

`SalesOrder` 记录订单编号、销售类型、会员、收银员工、预订关联及完成时间；`SalesOrderItem` 记录商品、数量和实际成交单价，其联合主码为 `(order_id,product_id)`。订单头与明细分离，使一笔现货订单能够容纳多种商品，同时避免每条明细重复保存会员和收银员工。

普通顾客的 `member_id` 可以为空。现货订单为 SPOT，预订关联为空；预售提货订单为 PRESALE，必须关联会员和预订。我们用 `(reservation_id,member_id)` 复合外键保证提货会员与预订会员一致，并用过滤唯一索引限制一条非空预订最多形成一笔订单：

```
CREATE UNIQUE INDEX UX_SalesOrder_ReservationID
ON dbo.SalesOrder(reservation_id)
WHERE reservation_id IS NOT NULL;
```

这样既限制重复提货订单，也允许多笔不关联预订的普通现货订单。`order_no` 是订单业务候选码；上述索引表达的是非空预订关联的条件唯一性。

成交单价必须保存在明细中，不能直接取商品当前售价，否则后续调价会改变历史销售统计。订单总金额通过 `SUM(quantity*unit_price)` 计算，不再重复保存。样例 SO004 包含甘雨立牌 1 件和派蒙徽章 2 件，总金额为 $68 + 2 \times 28 = 124$ 元。预订表也不重复保存最终成交价和提货时间，它们分别由销售明细和订单完成时间表达。

3.4 库存状态与预售决策数据

`Inventory.product_id` 同时是主码和商品外码，保证同一商品最多存在一条当前库存记录；商品与库存的结构关系为 1:0..1。正常经营时为商品初始化库存，表结构本身不强制每个商品都有库存记录。

库存只保存实际数量、已到货商品的预留数量和补货警戒值，并约束：

$$0 \leq \text{reserved_qty} \leq \text{on_hand_qty}, \quad \text{available_qty} = \text{on_hand_qty} - \text{reserved_qty}.$$

例如在库 5 件、预留 2 件时，可售库存为 3 件；若其中一件预留因逾期释放，在库量不变，可售库存变为 4 件。这种结构无需另建“预售库存”和“现货库存”两套记录。

预售决策字段采用不同原则：`decision_demand_qty` 保存采购决策发生时的需求快照，`extra_stock_qty` 保存店长的额外备货选择，`planned_purchase_qty` 保存最终采购计划。历史快照不会随取消或提货而改写，因此与查询实时计算的 `ACTIVE` 需求可以不同。可计算的当前状态尽量不重复保存，但具有历史决策意义的信息需要保留。

4 数据库建立与数据修改实验

4.1 执行顺序与样例数据

完整执行链为：

00 建库/复用 → 01 重建表 → 02 样例装载 → 03 CRUD
→ 04 查询 → 05 视图 → 06 约束验证 → 07 权限验证

建表按照先被引用、后引用的顺序进行，重建时先删除视图，再按相反依赖顺序删除业务表。00 脚本在数据库不存在时创建，存在时复用；01 脚本负责清空并重建本项目结构。这里的重复运行是课程样例的确定性重建，不是保留业务数据的增量更新。05 使用 `CREATE OR ALTER VIEW`，06 和 07 对补充约束、角色及测试用户设置存在性判断。

样例数据覆盖全部 15 张表，其中商品 6 条、库存 6 条、会员 5 条、销售订单 6 条、销售明细 8 条、预售活动 2 条、预订 4 条、采购订单 3 条、采购明细 5 条。它们体现了非会员购买、会员多商品订单、预售提货、采购后取消和开放中预售等情况。我们没有只插入彼此独立的元组，而是让商品、会员、员工、预订、销售和采购记录形成可供查询的关联。

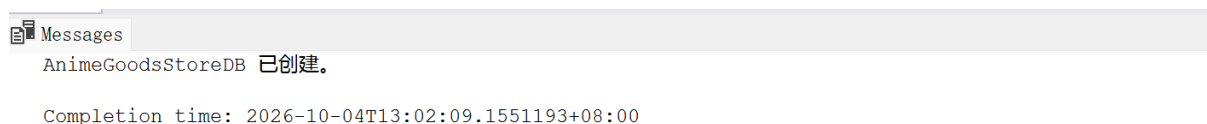


图 3: SSMS 中的数据库创建成功消息

4.2 典型 CRUD 流程与结果

03 脚本使用 9001 系列独立编号。商品实验先新增测试商品与库存，查询确认后修改售价和商品名称；库存由 10 件改为 15 件，警戒值由 3 改为 5，并同步修改 `updated_at`。查询使用相同 `WHERE` 条件确认目标，避免无条件更新。图4展示了这些操作的前后变化。

订单实验先创建 `CREATED` 订单及明细，将数量从 1 改为 2，按历史单价 68 元得到明细金额 136 元；确认明细后再将订单更新为 `COMPLETED` 并记录完成时间。该顺序对应“先确认交易内容，再完成结账”，避免演示中在订单完成后继续调整明细。

1	product_id	product_code	product_name	sale_price	product_status	on_hand_qty	reserved_qty	available_qty	reorder_point
1	9001	P_TEST	CRUD测试商品	20.00	ON_SALE	10	0	10	3
1	product_id	product_code	product_name	category_id	ip_id	character_id	sale_price	product_status	created_at
1	9001	P_TEST	CRUD测试商品	1	NULL	NULL	20.00	ON_SALE	2026-10-04 13:10:12.5945956
1	product_id	product_code	product_name	category_id	ip_id	character_id	sale_price	product_status	created_at
1	9001	P_TEST	CRUD测试商品 (修改后)	1	NULL	NULL	22.00	ON_SALE	2026-10-04 13:10:12.59459
1	product_id	on_hand_qty	reserved_qty	reorder_point	updated_at				
1	9001	10	0	3	2026-10-04 13:10:12.6018033				
1	product_id	on_hand_qty	reserved_qty	reorder_point	updated_at		available_qty		
1	9001	15	0	5	2026-10-04 13:10:12.9333711	15			

图 4: 商品信息修改与库存由 10 件改为 15 件的运行结果

删除时先删除订单明细再删除订单，商品删除前先清理对应库存，体现外键对依赖关系的保护。实验结束后清理临时编号。经人工运行和检查，商品、库存与订单相关增删改查均符合预期，正式样例可继续用于后续查询。完整步骤见《第三周脚本说明.md》及03_crud.sql。

5 经营查询与统计视图

5.1 查询覆盖与统计口径

查询围绕经营问题组织，涵盖连接、聚合、分组、HAVING 和子查询。销量与销售金额只统计 COMPLETED 订单，金额按历史成交单价计算；订单详情则保留订单类型和状态，便于查看业务记录。

5.2 连接与聚合的典型分析

Q1 连接订单、明细、商品和员工，对会员使用 LEFT JOIN。原因是非会员购买时member_id=NULL，若使用 INNER JOIN，SO001、SO006 等正常订单会消失。多商品订单出现多行明细则是正常结果，不能将明细行数直接当成订单数。

Q5 从会员出发，将完成状态条件放入 LEFT JOIN 的 ON 子句，使未消费会员也保留。核心实现如下：

编号	业务问题	主要 SQL 方法
Q1	谁购买了什么，由谁收银，明细金额是多少？	多表 INNER JOIN、会员 LEFT JOIN。
Q2	各商品的累计销量与销售额是多少？	LEFT JOIN、CASE、SUM、GROUP BY。
Q3、Q4	各 IP、各商品类别的销售表现如何？	分类维度连接与分组聚合。
Q5	每位会员完成几笔订单、累计消费多少？	LEFT JOIN、COUNT DISTINCT、SUM。
Q6	哪些商品需要关注补货？	可售库存计算与阈值筛选。
Q7	各预售活动当前还有多少有效需求？	按 ACTIVE 预订聚合，与快照对照。
Q8	哪些商品累计销量不少于 2 件？	HAVING 筛选聚合结果。
Q9	哪些消费会员的累计消费高于平均值？	派生表、子查询与 AVG。

表 5: 9 条经营查询的总体覆盖

```

SELECT m.member_no,
       COUNT(DISTINCT so.order_id) AS completed_order_count,
       SUM(COALESCE(soi.quantity * soi.unit_price, 0))
         AS total_consumption
FROM   dbo.Member AS m
LEFT JOIN dbo.SalesOrder AS so
      ON m.member_id = so.member_id
      AND so.order_status = 'COMPLETED'
LEFT JOIN dbo.SalesOrderItem AS soi
      ON so.order_id = soi.order_id
GROUP BY m.member_no;

```

林晓的 SO003、SO004 共 2 笔订单，但连接后有 3 条明细；COUNT DISTINCT 得到正确订单数，累计消费为 $129+68+2\times 28 = 253$ 元。陈雨消费 90 元，周琪消费 74 元，另外两位会员为 0。Q9 的平均对象是**有实际消费的会员**，平均累计消费为 $(253+90+74)/3 = 139$ 元，因此筛选出林晓；这与对全部注册会员求平均是不同的业务口径。

商品、IP 和类别统计提供了另一组一致性检查：样例累计售出 10 件，销售额 530 元。原神相关商品 7 件、327 元，星穹铁道相关商品 2 件、164 元，初音未来相关商品 1

件、39 元，分类总额与商品总额一致。运行结果见图5。

product_id	product_code	product_name	total_sales_qty	total_sales_amount
2	P002	原神群像徽章套装	3	135.00
1	P001	甘雨亚克力立牌	2	136.00
4	P004	派蒙徽章	2	56.00
3	P003	三月七毛绒玩偶	1	129.00
5	P005	星穹铁道群像色纸	1	35.00
6	P006	初音未来亚克...	1	39.00

ip_id	ip_name	total_sales_qty	total_sales_amount
1	原神	7	327.00
2	崩坏：星穹铁道	2	164.00
3	初音未来	1	39.00

图 5: 商品与 IP 维度的销量、销售额统计结果

Q7 区分当前仍需履约的需求和当时采购决策：PS001 的历史需求为 2 件，额外备货 1 件、计划采购 3 件；一条预订已提货、另一条已取消，当前 ACTIVE 需求为 0。PS002 两条有效预订分别为 1 件、2 件，当前需求为 3 件，采购快照尚未填写。保留两类信息，才能解释经营过程中的需求变化。Q8 则对分组后的SUM(quantity) 使用 HAVING，得到 P001、P002、P004 三种销量不少于 2 件的商品。经人工核验，各查询结果与样例业务含义一致。

5.3 视图封装与复用

我们将 4 类高频查询封装为视图：vw_OrderDetail 用于订单详情，vw_ProductSalesSummary 用于商品销量及销售额，vw_InventoryStatus 用于库存状态，vw_MemberConsumptionSummary 用于会员消费统计。视图复用基础表数据和计算规则，也为不同员工提供适合职责的查询入口。

库存状态视图实时计算可售库存，先判断数量为 0 的 OUT_OF_STOCK，再判断达到或低于警戒值的 LOW_STOCK，其余为 NORMAL。样例 P003 实际库存 2 件、无预留、警戒值 3 件，被标记为 LOW_STOCK，其他商品为 NORMAL，见图6。

product_id	product_code	product_name	on_hand_qty	reserved_qty	available_qty	reorder_point	stock_status
3	P003	三月七毛绒玩偶	2	0	2	3	LOW_STOCK
2	P002	原神群像徽...	5	0	5	4	NORMAL
5	P005	星穹铁道群...	6	0	6	3	NORMAL
1	P001	甘雨亚克力立牌	8	0	8	5	NORMAL
6	P006	初音未来亚...	9	0	9	4	NORMAL
4	P004	派蒙徽章	12	0	12	5	NORMAL

图 6: 库存状态视图对可售库存与补货警戒值的判断

视图查询结果经人工检查与原查询一致。详细查询与视图设计见《第四周设计说明.md》，完整实现分别位于04_query.sql 和05_view.sql。

6 完整性约束与正反例验证

6.1 约束总体设计与测试方法

建表阶段使用 PRIMARY KEY 保证内部标识唯一, UNIQUE 保护业务编号等候选码, FOREIGN KEY 维护引用关系, NOT NULL 和 DEFAULT 表达必填及初始状态, CHECK 限制金额、数量、枚举状态及同一行内字段关系。第四周补充“已退订金不超过实付订金”, 并用 2 组合法正例和 14 组非法反例检查实际效果。

测试	验证内容	预期机制
正例 P1、P2	合法商品与库存; 合法订金预订。	成功插入, 随后回滚。
T1-T4	负售价、预留超过在库、重复商品编号、不存在的商品类别。	CHECK、UNIQUE、外键。
T5、T6	IP 与角色组合错误; 退款超过订金。	复合外键、CHECK。
T7-T9	订金高于预售价、提货期限过早、取消缺少取消时间。	金额、时间、状态字段约束。
T10、T11	提货会员与预订不一致; 完成订单缺少交易时间。	复合外键、状态时间约束。
T12-T14	采购状态与验收信息不一致; 预计或实际到货早于下单。	验收信息与时间约束。

表 6: 完整性正反例测试的覆盖范围

测试使用独立编号, 在事务中执行并回滚, 避免污染正式样例。反例通过 TRY/CATCH 捕获错误, 同时核对错误号和预期约束名: CHECK 与外键冲突通常检查 547, 重复商品编号检查 2601 或 2627。只有预期规则拒绝输入才输出 PASS, 不能把其他 SQL 错误误认为约束成功。这里的事务用于隔离测试数据, 不代表已经实现销售和采购的完整业务事务。

6.2 典型正反例及结果分析

合法商品与库存。正例 P1 插入合法商品, 并设置实际库存 10 件、预留 3 件、警戒值 5 件。商品分类、IP 及角色引用均有效, 库存满足数量关系, 数据库允许插入。图7展示正例以及部分反例的运行消息。

IP 与角色不一致。T5 使用ip_id=1 (原神) 和character_id=3 (三月七)。两个

编号分别存在，但组合不属于同一角色记录，因此FK_Product_Character_IP 拒绝插入。该例说明参照完整性不仅是“引用存在”，还可以表达关联身份的一致性。

退款超过实付订金。 T6 设置deposit_paid=20、deposit_refunded=30，由CK_PresaleReservation_RefundNotExceedPaid 拒绝。这是同一行金额关系，可直接用CHECK 强制；截止前后是否退款则需读取活动截止时间，安排在后续业务过程中处理。

预售订单会员不匹配。 T10 引用属于会员 2 的预订 R002，却把订单会员填写为 1。FK_SalesOrder_ReservationMember 对两字段组合检查并拒绝插入，避免为其他会员的预订建立提货订单。

```

===== Positive Test P1 =====

(1 row affected)

(1 row affected)
[PASS] P1: 合法商品和库存数据插入成功。
===== Positive Test P2 =====

(1 row affected)
[PASS] P2: 合法订金数据插入成功。
===== T1: 负数商品售价 =====

(0 rows affected)
[PASS] T1: 负数商品售价因 CK_Product_SalePrice 被正确拒绝。
错误号: 547

```

图 7: 合法数据通过与非法商品价格被约束拒绝的运行消息

小组已对完整测试进行人工审验，合法数据成功，T1-T14 均因相应预期规则被拒绝，结果符合预期；回滚后未留下临时测试记录。完整用例与约束名称见06_constraint.sql。这些约束能保护已覆盖的数据关系，但状态取值合法与状态迁移合法并不相同，后者仍需后续流程控制。

7 角色权限与正常越权测试

7.1 最小权限与角色映射

BusinessRole 保存员工的现实经营职责，SQL Server 数据库角色真正控制访问，两者不能等同。我们创建dbrole_sales_clerk、dbrole_inventory_manager 和dbrole_store_manager 三个角色，按对象和字段逐项授权，不直接赋予 db_owner 等全权限角色。

视图授权使员工不必获得所有底层表权限即可使用常见结果，例如销售店员通过库存状态视图查询可售数量，通过订单详情视图查看销售记录。列级 UPDATE 则限定其可以修改的字段。会员的“只能访问本人记录”属于后续应用身份和行范围控制，本阶段不直接为会员授予基础表权限；供应商也不建立登录角色。

数据或操作	销售店员	库存管理员	店长
商品资料	查询。	查询。	增删改查。
会员资料	查询、登记、修改。	不授权。	查询。
销售订单	新增；修改状态、时间； 按订单 ID 定位。	不授权。	查询。
销售明细	新增；修改数量和单价。	不授权。	查询。
库存	通过状态视图查询。	查询；修改数量、警戒 值和更新时间。	查询；修改警戒值。
预售	查询活动、预订；修改 预订状态。	不授权。	管理活动；查询预订。
采购	不授权。	查询；修改验收信息与 到货数量。	管理采购订单及明细。
员工与供应商	不授权。	不授权。	管理基础资料。

表 7: 主要对象与操作的角色权限概览

7.2 典型权限验证

为避免额外建立真实服务器登录，我们创建 3 个 WITHOUT LOGIN 数据库用户并加入对应角色，以 EXECUTE AS USER 切换上下文，测试后 REVERT，修改类测试回滚。

正常测试 P1 由销售店员创建订单及明细，然后将 CREATED 更新为 COMPLETED；P2 由库存管理员更新补货警戒值和时间；P3 由店长修改商品售价。越权测试 N1 尝试让销售店员修改采购订单，N2 让库存管理员读取会员数据，N3 让销售店员直接修改实际库存。测试同时覆盖“该做的操作能完成”和“不属于职责的操作被拒绝”。

```
EXECUTE AS USER = 'u_test_inventory';
EXEC sys.sp_executesql N'SELECT * FROM dbo.Member;';
-- Permission test code catches error 229 and checks Member.
REVERT;
```

N2 对应库存管理员无权访问会员个人资料，数据库返回 229 权限拒绝错误。N1、N3 执行越权修改同样会被访问控制阻止；UPDATE 中的目标定位也需要读取相关列，实际检查过程中可能首先遇到 SELECT 权限拒绝。测试脚本据错误号与目标对象确认操作确实被权限系统阻止。经人工审验，3 组正常测试与 3 组越权测试均符合预期，见图8。

权限实验中有一项有代表性的修正：起初销售店员已有 INSERT 和状态、时间列的 UPDATE 权限，但 P1 在 WHERE order_id=9301 定位订单时仍缺少 SELECT 权限。我们

```

===== P1: 销售店员正常销售操作 =====

(1 row affected)

(1 row affected)

(1 row affected)
[PASS] P1: 销售店员可以正常创建和完成销售订单。
===== P2: 库存管理员正常库存操作 =====

(1 row affected)
[PASS] P2: 库存管理员可以正常更新库存信息。
===== P3: 店长正常商品管理 =====

(1 row affected)
[PASS] P3: 店长可以正常管理商品信息。
===== N1: 销售店员越权修改采购订单 =====

(0 rows affected)
[PASS] N1: 销售店员修改 PurchaseOrder 因 UPDATE 权限不足被正确拒绝。
错误号: 229
错误信息: 拒绝了对对象 'PurchaseOrder' (数据库 'AnimeGoodsStoreDB', 架构 'dbo')的 SELECT 权限。
===== N2: 库存管理员越权读取会员信息 =====
[PASS] N2: 库存管理员读取 Member 因 SELECT 权限不足被正确拒绝。
错误号: 229
错误信息: 拒绝了对对象 'Member' (数据库 'AnimeGoodsStoreDB', 架构 'dbo')的 SELECT 权限。
===== N3: 销售店员越权修改库存 =====

(0 rows affected)
[PASS] N3: 销售店员修改 Inventory 因 UPDATE 权限不足被正确拒绝。
错误号: 229
错误信息: 拒绝了对对象 'Inventory' (数据库 'AnimeGoodsStoreDB', 架构 'dbo')的 SELECT 权限。

```

图 8: 三种角色的正常操作成功，越权操作被数据库拒绝

没有扩大为整个订单表的读取权限，而是补充：

```

GRANT SELECT (order_id)
ON OBJECT::dbo.SalesOrder TO dbrole_sales_clerk;

```

修正后 P1 正常完成。该问题说明最小权限不是单纯减少授权，还要考虑完整 SQL 操作所需的读取条件。完整授权和测试见 `07_role.sql`。

8 小组协作与 AI 辅助下的独立判断

8.1 成员分工与协同方式

小组成员为郑立峰（24325341）、李维政（24325143），均来自计算机学院计算机科学与技术专业。我们先共同讨论场景和实验安排，再采用实现、复核、修订的协作方式，避免仅由代码编写者判断自己的结果。

小组讨论记录保留了场景规模、预售取消、提货期限和采购规则等选择过程。各周文档明确设计，SQL 落实结构与操作，README 组织复现入口，AI 对话记录保留建议及人工修改，分工文档说明责任。这些材料与阶段报告各有用途，共同组成可理解、可运行的阶段提交。

环节	郑立峰	李维政
需求与安排	共同讨论场景、业务边界和阶段任务。	共同讨论场景、业务边界和阶段任务。
实现与检查	协同 AI 实现和修订第 1 至 4 周业务产出。	检查实现方向，对已有产出进行完整二次人工审核。
阶段整理	根据审核结果多轮修订代码与设计文档。	撰写 README 与协作文档。
报告与收尾	负责阶段报告编写与整理。	参与报告审阅、核对设计与实验表述。

表 8: 第 1 至 4 周小组分工安排

8.2 AI 建议、人工选择与验证

AI 主要辅助提出候选方案、整理数据结构、生成候选 SQL 和审查不一致。我们对业务含义、复杂度及实现方式作出选择，而不是将每条建议直接纳入项目。

预售的建模修正。早期方案曾把商品分为预售和现货两类。人工讨论认为同一商品可能先预售、到货后再现货销售，因此改为两种销售业务，并独立建立预售活动表。取消是否需要店铺批准、截止后是否退订金、逾期如何处理，也由小组确定后再落实到业务设计。

复杂度与数据冗余的取舍。对多 IP、多角色的中间表建议，我们选择暂不纳入，采用可空的商品 IP 与角色字段；对订单总金额、重复提货时间等字段，则采纳删除冗余的建议。这里既不是追求表越多越好，也不是一律少存字段，而是区分当前可计算值与历史交易、决策信息。

实现与运行检查。AI 协助生成建表顺序、查询和测试框架后，我们在本地执行脚本，对结果作人工核对。出现销售正常操作失败时，进一步分析 SQL 所需读权限，并选择只补 order_id 列的 SELECT。修改有明确问题依据，而不是为了让测试通过而直接授予全表权限。

8.3 整合审查与改进

阶段收尾时，我们将审查发现的问题分成“当前应修正”和“后续实现”两类。当前修正包括：调整 CRUD 顺序，先确认明细再完成订单；将反例判断由“执行失败”改为核对预期错误原因；补齐状态、时间及预售身份一致性约束；统一空库与已有库上的完整执行链。跨表库存联动、退款与状态机则写入待实现清单，避免文档将业务目标提前表述成已经自动实现。

会员消费聚合曾出现“消除了 NULL 值”的警告。我们将`COALESCE(SUM(...), 0)`调整为`SUM(COALESCE(..., 0))`，先将无明细的金额转换为 0 再聚合，保持业务含义并使输出更清晰。最终代码经组员复核和运行检验，阶段文档同步修订。上述过程体现了 AI 辅助提案、人工判断、实际运行及迭代修正之间的分工，详细记录见《小组讨论议题.md》及仓库的“第 1-4 周 ai 对话”目录。

9 实验总结与后续工作

本阶段将经营需求转化为 15 张关联数据表，在 SQL Server 中完成建库、样例装载和 CRUD，再通过 9 条查询、4 个视图、完整性正反例和角色权限验证形成数据库原型。商品、订单、预售与采购不再是孤立信息，能够支持订单追踪、销售统计、库存检查和需求分析。

实验中最重要认识是：设计应从业务语义出发。现货与预售的区分影响活动和订单结构；历史成交价与采购快照的保留服务于后续解释；复合外键保护的不只是编号存在，还包括身份匹配。与此同时，SQL 运行成功并不意味着业务解释正确，连接方式、聚合口径和权限条件都需要结合样例和职责逐项判断。

协作方面，方向讨论、二次人工审核和多轮修订帮助我们发现单人实现容易忽略的问题。AI 提高了候选设计和 SQL 整理效率，但对建议的取舍、业务边界的确定及运行结果的解释仍由小组承担。我们通过具体问题的修改理解了设计，而不是只获得一组能够执行的脚本。

下一阶段将继续使用现有经营场景，结合 ER 建模与规范化反思关系结构，并推进数据库应用。后续数据库编程与事务阶段重点实现销售完成时的库存检查和扣减、采购验收入库、预售预留与释放、订金退款判断以及合法状态迁移。会员本人数据的访问由应用身份和数据范围控制落实。相关规则已集中记录在《后续阶段待实现业务规则.md》，为后续任务提供清楚的衔接。

相关材料索引

课程要求见《第一周任务讲解.docx》至《第四周任务讲解.docx》。协同仓库 Database_exp 中，README.md 提供项目范围和复现方法，“第 1-4 周业务产出”目录保存每周设计文档及 00-07 SQL 脚本，results 保存运行截图，《小组分工.md》和《小组讨论议题.md》记录协作与 AI 使用过程。本报告图中的运行结果摘自既有 SSMS 截图，便于阅读而选取相应结果区域。